

Melek Mizher
MSDS 422
Lawrence Fulton
05/22/2022

Module 8 Assignment 1: Dogs vs. Cats Redux

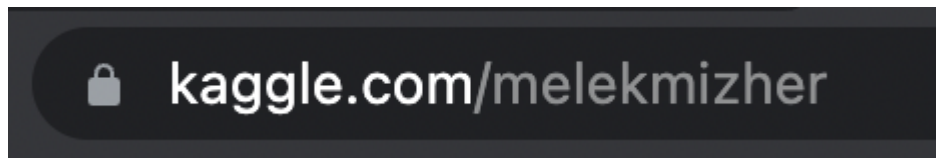
For the Dogs vs. Cats Redux Kaggle Competition, the Management/Research Question of interest is to develop a model that can successfully identify differences between cats and dogs to classify them correctly. I created three Neural Network models using Keras through Tensorflow. I was not able to run them through the entire dataset due to lacking wifi and my network connection was unstable. I was fitting the models on a subsample of the first 1000 training images of cats and the first 1000 for dogs. This model was then used to predict the first 500 test images. This was a fairly quick subsample to run, especially when condensed to a 50x50x3 image shape. In this case, the model offered an accuracy of 82.50%, which is not bad considering that a lot of information was lost in the process and that the amount of training data used to fit the model was very small. The Neural Network models used provide probabilities of each test image being a dog, which is what is asked from the evaluation details in the competition.

The first NN algorithm consisted of a Convolutional layer, followed by a MaxPooling layer, Flattening, and one Dense layer with ReLu activation before the Final Dense Sigmoid layer (as the end goal was a probability). The second NN algorithm had the same structure but used larger image shapes (200x200x3). The third CNN model is much more complex and had multiple convolutional layers intermingled with max-pooling and dropout layers; however, there were some issues with the input sizes that I was unable to resolve. I was also unable to use the model history in order to draw proper goodness of fit metrics. However, the code is provided below.

Since the dataset was very large, I used a system to iterate through the data faster by using batches of 64 images at once for both the training and testing sets. This also made it so that the entire dataset was not in memory at one given time so that the computer could process it. For the entire CNN to be fit on the training data, close to 3 hours were required. However, this crashed multiple times as my computer was connected to a mobile hotspot. Most of the work was done through Google collab as I imagined that my computer would not be able to handle the fitting process. After multiple resets and deletion of my /tmp/ folder in collab, I transferred my work back into my laptop and ran it using a regular jupyter-notebook as I had done in previous submissions. I am currently leaving it running overnight on both collab and the notebook and I will update this assignment with more information soon. All the setup of the data frames and CSV files is prepared into the code so all that will be left to do is submit the file.

I also had issues incorporating the Kaggle API key into the Google collab. I followed multiple tutorials online but it took me over a day of toying with Google collab to figure out the connection to the Kaggle API until I finally decided to just upload the dataset from my laptop instead.

Appendix A: Kaggle Username and CNN Submission Score (N/A. Accuracy: 82.50%)



Appendix B: Python Notebook and Data Visualization/Data Tables

Dogs vs. Cats Redux Mizher

Initial Imports

In [1]:

```
import pandas as pd
import numpy as np
from numpy import std, absolute, mean
import seaborn as sns
import sklearn as sk
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
import datetime
import tensorflow as tf
from tensorflow.python.client import import device_lib #GPU Check
```

In [2]:

```
import tensorflow.keras #keras
from tensorflow.keras import layers
from tensorflow.keras import Sequential, Input, Model
from tensorflow.keras.layers import Dense, Dropout, Flatten, Input, Add,
Activation, ZeroPadding2D
from tensorflow.keras.layers import BatchNormalization, Conv2D,
AveragePooling2D, MaxPooling2D, GlobalMaxPooling2D
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau,
ModelCheckpoint #use for early stopping
from tensorflow.keras.initializers import glorot_uniform, he_uniform #to
initialize random weights for filters
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.preprocessing import image as image_utils
from tensorflow.keras.applications.imagenet_utils import preprocess_input,
decode_predictions
from tensorflow.keras.models import Sequential, Model, load_model
from tensorflow.keras import utils
from tensorflow.keras.utils import get_file, plot_model, to_categorical,
model_to_dot
```

More than one Output per Box

In [3]:

```
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

In [4]:

```
%pwd
```

Out[4]:

'/Users/melekmizher/Desktop'

Upload files (First method failed; uploaded zips to google drive instead.

In [5]:

```
from google.colab import files
test = files.upload()
```

ModuleNotFoundError Traceback (most recent call last)

Input In [5], in <cell line: 1>()

```
----> 1 from google.colab import files
      2 test = files.upload()
```

ModuleNotFoundError: No module named 'google.colab'

In [6]:

```
from google.colab import files
train = files.upload()
```

ModuleNotFoundError Traceback (most recent call last)

Input In [6], in <cell line: 1>()

```
----> 1 from google.colab import files
      2 train = files.upload()
```

ModuleNotFoundError: No module named 'google.colab'

In [7]:

```
import zipfile
import os, io
zip_ref = zipfile.ZipFile("/content/drive/MyDrive/Colab Notebooks/Dogs vs.
Cats Redux Files/test.zip", 'r') #Opens the zip file in read mode
zip_ref.extractall('/tmp') #Extracts the files into the /tmp folder
zip_ref.close()
```

FileNotFoundError Traceback (most recent call last)

Input In [7], in <cell line: 3>()

```
  1 import zipfile
  2 import os, io
----> 3 zip_ref = zipfile.ZipFile("/content/drive/MyDrive/Colab Notebooks/Dogs vs. Cats Redux
Files/test.zip", 'r') #Opens the zip file in read mode
      4 zip_ref.extractall('/tmp') #Extracts the files into the /tmp folder
```

```
5 zip_ref.close()
```

File /opt/anaconda3/envs/tensorflow/lib/python3.10/zipfile.py:1249, in ZipFile.__init__(self, file, mode, compression, allowZip64, compresslevel, strict_timestamps)

```
1247 while True:
1248     try:
-> 1249         self.fp = io.open(file, filemode)
1250 except OSError:
1251     if filemode in modeDict:
```

FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/Colab Notebooks/Dogs vs. Cats Redux Files/test.zip'

In [8]:

```
import zipfile
import os, io
zip_ref = zipfile.ZipFile("/content/drive/MyDrive/Colab Notebooks/Dogs vs.
Cats Redux Files/train.zip", 'r') #Opens the zip file in read mode
zip_ref.extractall('/tmp') #Extracts the files into the /tmp folder
zip_ref.close()
```

FileNotFoundError Traceback (most recent call last)

Input In [8], in <cell line: 3>()

```
1 import zipfile
2 import os, io
----> 3 zip_ref = zipfile.ZipFile("/content/drive/MyDrive/Colab Notebooks/Dogs vs. Cats Redux
Files/train.zip", 'r') #Opens the zip file in read mode
4 zip_ref.extractall('/tmp') #Extracts the files into the /tmp folder
5 zip_ref.close()
```

File /opt/anaconda3/envs/tensorflow/lib/python3.10/zipfile.py:1249, in ZipFile.__init__(self, file, mode, compression, allowZip64, compresslevel, strict_timestamps)

```
1247 while True:
1248     try:
-> 1249         self.fp = io.open(file, filemode)
1250 except OSError:
1251     if filemode in modeDict:
```

FileNotFoundError: [Errno 2] No such file or directory: '/content/drive/MyDrive/Colab Notebooks/Dogs vs. Cats Redux Files/train.zip'

In [6]:

```
# plot cat photos from the dogs vs cats dataset
from matplotlib import pyplot
from matplotlib.image import imread
# define location of dataset
```

```

folder = '/tmp/train/'
# plot first few images
for i in range(9):
    # define subplot
    pyplot.subplot(330 + 1 + i)
    # define filename
    filename = folder + 'cat.' + str(i) + '.jpg'
    # load image pixels
    image = imread(filename)
    # plot raw pixel data
    pyplot.imshow(image)
# show the figure
pyplot.show()

```

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f570eed0>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f507e1d0>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f5048b90>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f50a2610>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f4fb3250>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f50dded0>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f56ba410>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f56e0950>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f50d07d0>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f5623250>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f562c710>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f5659bd0>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f55e1f90>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f561b590>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f55a3ad0>

Out [6] :

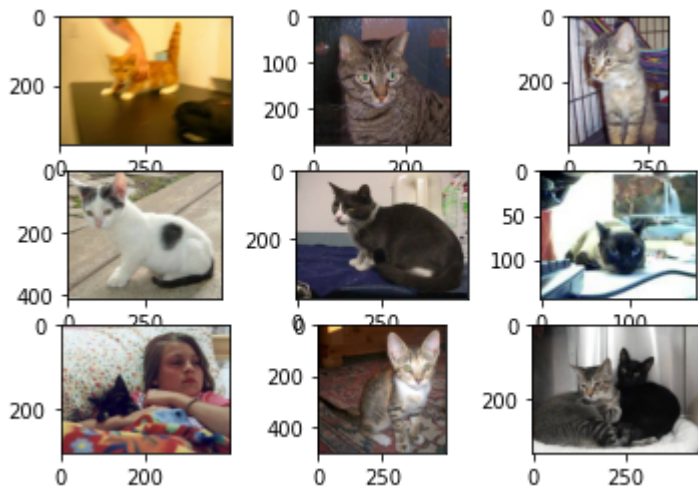
<matplotlib.image.AxesImage at 0x7fa0f55d1f50>

Out [6] :

<matplotlib.axes._subplots.AxesSubplot at 0x7fa0f5566450>

Out [6] :

<matplotlib.image.AxesImage at 0x7fa0f56c17d0>



In [45]:

```
import os.path
from os import path

if path.exists('/tmp/train/cats/') == False:
    os.mkdir('/tmp/train/cats/')
if path.exists('/tmp/train/dogs/') == False:
    os.mkdir('/tmp/train/dogs/')

if path.exists('/tmp/test/test/') == False:
    os.mkdir('/tmp/test/test/')
```

In [27]:

```
import os, shutil
#move picture.png from /some/dir/ to /another/dir/
for i in range(0,12500):
    shutil.move('/tmp/train/cat.' + str(i) + '.jpg', '/tmp/train/cats/cat.' +
str(i) + '.jpg')
    shutil.move('/tmp/train/dog.' + str(i) + '.jpg', '/tmp/train/dogs/dog.' +
str(i) + '.jpg')
```

In [50]:

```
for i in range(1,12501):
    shutil.move('/tmp/test/' + str(i) + '.jpg', '/tmp/test/test/' + str(i) +
'.jpg')
```

In [28]:

```
import pandas as pd
train = pd.DataFrame([], columns=["Filename", "number"])
train
```

Out[28]:

Filename	number
----------	--------

In [29]:

```
import pandas as pd
folder = '/Users/melekmizher/Desktop/Dogs vs Cats Redux/train/dogs/'
```

```

names = []
names2 = []

for i in range(0,12500):
    # define filename
    filename = folder + 'dog.' + str(i) + '.jpg'
    # append file path to dataframe
    names.append(filename)

folder = '/Users/melekmizher/Desktop/Dogs vs Cats Redux/train/cats/'

for i in range(0,12500):
    # define filename
    filename = folder + 'cat.' + str(i) + '.jpg'
    # append file path to dataframe
    names2.append(filename)

total = names + names2

```

In [30]:

```

train['Filename'] = total
train["number"] = list(range(12500)) + list(range(12500))
#train["id"] = list(range(25000))
y_train = pd.DataFrame(([0] * 12500) + ([1] * 12500), columns=["label"])

```

In [31]:

train

Out[31]:

	Filename	number
0	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	0
1	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	1
2	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	2
3	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	3
4	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	4
...	...	...
2499	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	12495
5		
2499	/Users/melekmizher/Desktop/Dogs vs Cats Redux/...	12496
6		

```
2499 /Users/melekmizher/Desktop/Dogs vs Cats Redux/... 12497
7
```

```
2499 /Users/melekmizher/Desktop/Dogs vs Cats Redux/... 12498
8
```

```
2499 /Users/melekmizher/Desktop/Dogs vs Cats Redux/... 12499
9
```

25000 rows × 2 columns

In [32]:

```
y_train
```

Out[32]:

	label
0	0
1	0
2	0
3	0
4	0
...	...
2499	1
5	
2499	1
6	
2499	1
7	
2499	1
8	
2499	1
9	

25000 rows × 1 columns

In [33]:

```
import pandas as pd
```

```
test = pd.DataFrame([], columns=["Filename", "number"])
test
```

Out[33]:

	Filename	number
--	----------	--------

In [34]:

```
import pandas as pd
folder = '/tmp/test/test/'
names = []

for i in range(0,12500):
    # define filename
    filename = folder + str(i) + '.jpg'
    # append file path to dataframe
    names.append(filename)
```

In [35]:

```
test['Filename'] = names
test["number"] = list(range(12500))
```

In [36]:

```
test
```

Out[36]:

	Filename	number
0	/tmp/test/test/0.jpg	0
1	/tmp/test/test/1.jpg	1
2	/tmp/test/test/2.jpg	2
3	/tmp/test/test/3.jpg	3
4	/tmp/test/test/4.jpg	4
...	...	...
12495	/tmp/test/test/12495.jpg	12495
12496	/tmp/test/test/12496.jpg	12496
12497	/tmp/test/test/12497.jpg	12497

```
1249      /tmp/test/test/12498.jpg    12498
      8
```

```
1249      /tmp/test/test/12499.jpg    12499
      9
```

12500 rows × 2 columns

In [37]:

```
from tensorflow.keras import Sequential, Input, Model
from tensorflow.keras.layers import Dense, Dropout, Flatten, Input, Add,
Activation, ZeroPadding2D
from tensorflow.keras.layers import BatchNormalization, Conv2D,
AveragePooling2D, MaxPooling2D, GlobalMaxPooling2D
from tensorflow.keras.optimizers import SGD
```

Model 1

In [41]:

```
# define cnn model
def define_model():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), activation='relu',
kernel_initializer='he_uniform', padding='same', input_shape=(200, 200, 3)))
    model.add(MaxPooling2D((2, 2)))
    model.add(Flatten())
    model.add(Dense(128, activation='relu', kernel_initializer='he_uniform'))
    model.add(Dense(1, activation='sigmoid'))
    # compile model
    opt = SGD(learning_rate=0.001, momentum=0.9)
    model.compile(optimizer=opt, loss='binary_crossentropy',
metrics=['accuracy'])
    return model
```

In [42]:

```
model = define_model()
```

In [43]:

```
# create data generator
from tensorflow.keras.preprocessing.image import ImageDataGenerator
datagen = ImageDataGenerator(rescale=1.0/255.0)
```

Full Sample

In [98]:

```
# prepare iterators
train_it = datagen.flow_from_directory('/Users/melekmizher/Desktop/Dogs vs
Cats Redux/train/',
    class_mode='binary', batch_size=64, target_size=(200, 200))
test_it = datagen.flow_from_directory('/Users/melekmizher/Desktop/Dogs vs
Cats Redux/test/',
```

```
class_mode='binary', batch_size=64, target_size=(200, 200))
```

Found 25000 images belonging to 2 classes.

Found 12500 images belonging to 1 classes.

Subsample

In [44]:

```
# prepare iterators
train_it = datagen.flow_from_directory('/Users/melekmizher/Desktop/Dogs vs
Cats Redux/Subsample/train/',
    class_mode='binary', batch_size=64, target_size=(200, 200))
test_it = datagen.flow_from_directory('/Users/melekmizher/Desktop/Dogs vs
Cats Redux/Subsample/test/',
    class_mode='binary', batch_size=64, target_size=(200, 200))
```

Found 2000 images belonging to 2 classes.

Found 500 images belonging to 1 classes.

In [45]:

```
# fit model
history = model.fit(train_it, steps_per_epoch=len(train_it),
    validation_data=test_it, validation_steps=len(test_it), epochs=20,
    verbose=0)
```

In [73]:

```
y_pred = model.predict(test_it)
```

In [51]:

```
print(history)
<keras.callbacks.History object at 0x17e82edd0>
```

In [52]:

```
pd.DataFrame(model.history).plot(figsize = (8 , 5))
plt.grid(True)
plt.gca().set_ylim(0, 1) # set the vertical range to [0-1]
plt.show()
```

ValueError Traceback (most recent call last)

Input In [52], in <cell line: 1>()

```
----> 1 pd.DataFrame(model.history).plot(figsize = (8 , 5))
      2 plt.grid(True)
      3 plt.gca().set_ylim(0, 1) # set the vertical range to [0-1]
```

File /opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/pandas/core/frame.py:756, in

DataFrame.__init__(self, data, index, columns, dtype, copy)

753 # For data is scalar

754 else:

755 if index is None or columns is None:

--> 756 raise ValueError("DataFrame constructor not properly called!")

758 # Argument 1 to "ensure_index" has incompatible type "Collection[Any]";

759 # expected "Union[Union[Union[ExtensionArray, ndarray],

```
760 # Index, Series], Sequence[Any]]"
```

```
761 index = ensure_index(index) # type: ignore[arg-type]
```

ValueError: DataFrame constructor not properly called!

In [74]:

```
y_pred = pd.DataFrame(y_pred) #Turn to dataframe so that it can be
concatenated
```

```
y_pred = y_pred.rename({0:"label"},axis=1) #Rename so that it can be
evaluated
```

```
y_pred["id"] = y_pred.index
```

```
y_pred["id"] = y_pred["id"].apply(lambda x : x+1)
```

```
NN_Submission = pd.concat([y_pred.id, y_pred.label], axis=1)
```

```
NN_Submission.to_csv('submission.csv', index=False)
```

In [76]:

```
NN_Submission
```

Out[76]:

	id	label
0	1	0.10474 0
1	2	0.49401 3
2	3	0.56899 8
3	4	0.10697 6
4	5	0.47573 8
...	...	...
49	49	0.73138
5	6	7
49	49	0.42651
6	7	7

```
49 49 0.67262
7 8 9
```

```
49 49 0.49946
8 9 1
```

```
49 50 0.61090
9 0 7
```

500 rows × 2 columns

Resize Subsample images to small format for model.

In [66]:

```
# prepare iterators
train_it = datagen.flow_from_directory('/Users/melekmizher/Desktop/Dogs vs
Cats Redux/Subsample/train/',
    class_mode='binary', batch_size=64, target_size=(50, 50))
test_it = datagen.flow_from_directory('/Users/melekmizher/Desktop/Dogs vs
Cats Redux/Subsample/test/',
    class_mode='binary', batch_size=64, target_size=(50, 50))
```

Found 2000 images belonging to 2 classes.

Found 500 images belonging to 1 classes.

Model 2

In [*]:

```
# define cnn model
def define_model():
    model = Sequential()
    model.add(Conv2D(32, (3, 3), activation='relu',
kernel_initializer='he_uniform', padding='same', input_shape=(200, 200, 3)))
    model.add(MaxPooling2D((2, 2)))
    model.add(Flatten())
    model.add(Dense(64, activation='relu', kernel_initializer='he_uniform'))
    model.add(Dense(1, activation='sigmoid'))
    # compile model
    opt = SGD(learning_rate=0.001, momentum=0.9)
    model.compile(optimizer=opt, loss='binary_crossentropy',
metrics=['accuracy'])
    return model
```

In [*]:

```
model = define_model()
```

In [*]:

```
# fit model
history3 = model.fit(train_it, steps_per_epoch=len(train_it),
```

```
validation_data=test_it, validation_steps=len(test_it), epochs=20,
verbose=0)
```

In [*]:

```
y_pred = model.predict(test_it)
```

In [*]:

```
y_pred = np.argmax(y_pred,axis=1)
y_pred = pd.DataFrame(y_pred) #Turn to dataframe so that it can be
concatenated
y_pred = y_pred.rename({0:"label"},axis=1) #Rename so that it can be
evaluated
```

```
y_pred["id"] = y_pred.index
y_pred["id"] = y_pred["id"].apply(lambda x : x+1)
```

```
NN2_Submission = pd.concat([y_pred.id, y_pred.label], axis=1)
```

```
NN2_Submission.to_csv('submission.csv', index=False)
```

In [*]:

```
import numpy
from numpy import mean, std
cvscores = []
# evaluate the model
scores = model.evaluate(train_it, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
cvscores.append(scores[1] * 100)
print("%.2f%% (+/- %.2f%%)" % (numpy.mean(cvscores), numpy.std(cvscores)))
```

Model 3

In [102]:

```
# define cnn model
def define_model():
    model = Sequential()
    model.add(Conv2D(filters=64, kernel_size=(3, 3),activation='relu',
padding="same", input_shape=input_shape))
    model.add(MaxPooling2D(pool_size=(2, 2), strides=(2,2)))
    model.add(Dropout(0.2))
    model.add(Conv2D(filters=128, kernel_size=(3, 3),activation='relu',
padding="same"))
    model.add(MaxPooling2D(pool_size=(2, 2), strides=(2,2)))
    model.add(Dropout(0.25))
    model.add(Conv2D(filters=256, kernel_size=(3, 3),activation='relu',
padding="same"))
    model.add(MaxPooling2D(pool_size=(2, 2), strides=(2,2)))
    model.add(Dropout(0.3))
    model.add(Conv2D(filters=512, kernel_size=(3, 3),activation='relu',
padding="same"))
    model.add(MaxPooling2D(pool_size=(2, 2), strides=(2,2)))
```

```

model.add(Dropout(0.35))
model.add(Conv2D(filters=512, kernel_size=(3, 3), activation='relu',
padding="same"))
model.add(MaxPooling2D(pool_size=(2, 2), strides=(2,2)))
model.add(Dropout(0.4))
model.add(Flatten())
model.add(Dense(4096, activation='relu'))
model.add(Dense(4096, activation='relu'))
model.add(Dense(num_classes, activation='softmax')) # compile model
opt = SGD(lr=0.001, momentum=0.9)
model.compile(optimizer=opt, loss='binary_crossentropy',
metrics=['accuracy'])
return model

```

In [103]:

```

# fit model
history3 = model.fit(train_it, steps_per_epoch=len(train_it),
validation_data=test_it, validation_steps=len(test_it), epochs=20,
verbose=0)

```

InvalidArgumentError Traceback (most recent call last)

Input In [103], in <cell line: 2>():

```

1 # fit model
----> 2 history3 = model.fit(train_it, steps_per_epoch=len(train_it),
3 validation_data=test_it, validation_steps=len(test_it), epochs=20, verbose=0)

```

File

/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/utils/traceback_utils.py:67, in filter_traceback.<locals>.error_handler(*args, **kwargs)

```

65 except Exception as e: # pylint: disable=broad-except
66     filtered_tb = _process_traceback_frames(e.__traceback__)
--> 67     raise e.with_traceback(filtered_tb) from None
68 finally:
69     del filtered_tb

```

File

/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/tensorflow/python/eager/execute.py:58, in quick_execute(op_name, num_outputs, inputs, attrs, ctx, name)

```

56 try:
57     ctx.ensure_initialized()
--> 58     tensors = pywrap_tfe.TFE_Py_Execute(ctx._handle, device_name, op_name,
59 inputs, attrs, num_outputs)
60 except core._NotOkStatusException as e:
61     if name is not None:

```

InvalidArgumentError: Incompatible shapes: [64,1] vs. [1024,1]

[[node binary_crossentropy/logistic_loss/mul
(defined at
/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/backend.py:5149)
]] [Op:__inference_train_function_19640]

Errors may have originated from an input operation.

Input Source operations connected to node binary_crossentropy/logistic_loss/mul:

In[0] sequential_5/dense_11/BiasAdd (defined at

/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/layers/core/dense.py:194)

In[1] ExpandDims (defined at

/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/engine/compile_utils.py:676)

Operation defined at: (most recent call last)

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/runpy.py", line 191, in
```

```
_run_module_as_main
```

```
>>> msg = "%s: %s" % (sys.executable, exc)
```

```
>>>
```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/runpy.py", line 75, in _run_code
```

```
>>> fname = mod_spec.origin
```

```
>>>
```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel_launcher.py",  
line 12, in <module>
```

```
>>> if sys.path[0] == "":
```

```
>>>
```

```
>>> File
```

```
"opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/traitlets/config/application.py", line  
844, in launch_instance
```

```
>>> app = cls.instance(**kwargs)
```

```
>>>
```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/kernelapp.py",  
line 697, in start
```

```
>>> from ipykernel.trio_runner import TrioRunner
```

```
>>>
```

```
>>> File
```

```
"opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/tornado/platform/asyncio.py", line  
195, in start
```

```
>>> old_loop = None # type: ignore
```

```
>>>
```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/asyncio/base_events.py", line 594, in  
run_forever
```

```
>>> old_asyncgen_hooks = sys.get_asyncgen_hooks()
```

```
>>>
```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/asyncio/base_events.py", line 1860, in  
_run_once
```

```
>>> event_list = self._selector.select(timeout)
```

```
>>>
```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/asyncio/events.py", line 80, in _run
```

```
>>> self._context.run(self._callback, *self._args)
```

```
>>>
```

```
>>> File
```

```
"opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/kernelbase.py", line 502,  
in dispatch_queue
```

```
>>> await self.process_one()
```

```

>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/kernelbase.py", line 488,
in process_one
>>>     t, dispatch, args = self.msg_queue.get_nowait()
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/kernelbase.py", line 375,
in dispatch_shell
>>>     return
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/kernelbase.py", line 693,
in execute_request
>>>     metadata = self.init_metadata(parent)
>>>
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/ipkernel.py",
line 355, in do_execute
>>>     preprocessing_exc_tuple=preprocessing_exc_tuple,
>>>
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/ipykernel/zmqshell.py",
line 528, in run_cell
>>>     return super().run_cell(*args, **kwargs)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/IPython/core/interactiveshell.py",
line 2863, in run_cell
>>>     result = self._run_cell(
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/IPython/core/interactiveshell.py",
line 2885, in _run_cell
>>>     raw_cell,
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/IPython/core/async_helpers.py",
line 129, in _pseudo_sync_runner
>>>     coro.send(None)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/IPython/core/interactiveshell.py",
line 3080, in run_cell_async
>>>     except self.custom_exceptions as e:
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/IPython/core/interactiveshell.py",
line 3277, in run_ast_nodes
>>>     elif interactivity == 'all':
>>>

```

```

>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/IPython/core/interactiveshell.py",
line 3366, in run_code
>>> if async_:
>>>
>>> File
"/var/folders/5m/ct1bwh6d20bfbgg2x0908mjm0000gn/T/ipykernel_23107/2016967770.py", line 3,
in <cell line: 2>
>>> validation_data=test_it, validation_steps=len(test_it), epochs=20, verbose=0)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/utils/traceback_utils.py", line
60, in error_handler
>>> return fn(*args, **kwargs)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/engine/training.py", line
1168, in fit
>>> y=y,
>>>
>>> File
"/var/folders/5m/ct1bwh6d20bfbgg2x0908mjm0000gn/T/__autograph_generated_filezyrj5cq8.py"
, line 12, in tf__train_function
>>> retval_ = ag__.UndefinedReturnValue()
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/engine/training.py", line
866, in step_function
>>> data = next(iterator)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/engine/training.py", line
860, in run_step
>>> outputs = model.train_step(data)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/engine/training.py", line
808, in train_step
>>> y_pred = self(x, training=True)
>>>
>>> File
"/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/engine/compile_utils.py",
line 193, in __call__
>>> zip_args = (y_true, y_pred, sample_weight, self._losses, self._loss_weights,
>>>
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/losses.py", line
137, in __call__
>>> if tf.executing_eagerly():
>>>

```

```
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/losses.py", line
244, in call
>>>   ag_fn = tf.__internal__.autograph.tf_convert(self.fn,
tf.__internal__.autograph.control_status_ctx())
>>>
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/losses.py", line
1800, in binary_crossentropy
>>>   def _smooth_labels():
>>>
>>> File "/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/keras/backend.py", line
5149, in binary_crossentropy
>>>   if from_logits:
>>>
```

In [85]:

```
import numpy
from numpy import mean, std
cvscores = []
# evaluate the model
scores = model.evaluate(train_it, verbose=0)
print("%s: %.2f%%" % (model.metrics_names[1], scores[1]*100))
cvscores.append(scores[1] * 100)
print("%.2f%% (+/- %.2f%%)" % (numpy.mean(cvscores), numpy.std(cvscores)))
accuracy: 82.50%
82.50% (+/- 0.00%)
```

In [97]:

```
from sklearn.metrics import roc_curve, roc_auc_score
import matplotlib.pyplot as pyplot

# predict probabilities
y_prob = model.predict(test_it)
# plot no skill roc curve
pyplot.plot([0, 1], [0, 1], linestyle='--', label='No Skill')
# calculate roc curve for model
roc_auc = roc_auc_score(test_it, y_prob)
# plot model roc curve
pyplot.plot(roc_auc, marker='.', label='Logistic')
# axis labels
pyplot.xlabel('False Positive Rate')
pyplot.ylabel('True Positive Rate')
# show the legend
pyplot.legend()
# show the plot
pyplot.show()
```

Out [97]:

```
[<matplotlib.lines.Line2D at 0x1c06d3280>]
```

ValueError Traceback (most recent call last)

Input In [97], in <cell line: 9>():

```
7 pyplot.plot([0, 1], [0, 1], linestyle='--', label='No Skill')
8 # calculate roc curve for model
--> 9 roc_auc = roc_auc_score(test_it, y_prob)
10 # plot model roc curve
11 pyplot.plot(roc_auc, marker='.', label='Logistic')
```

File

/opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/sklearn/metrics/_ranking.py:544, in roc_auc_score(y_true, y_score, average, sample_weight, max_fpr, multi_class, labels)

```
363 def roc_auc_score(
364     y_true,
365     y_score,
366     ...,
367     labels=None,
368 ):
369     """Compute Area Under the Receiver Operating Characteristic Curve (ROC AUC)
370     from prediction scores.
371
372     (...)
373     array([0.81..., 0.84..., 0.93..., 0.87..., 0.94...])
374     """
--> 544     y_type = type_of_target(y_true)
545     y_true = check_array(y_true, ensure_2d=False, dtype=None)
546     y_score = check_array(y_score, ensure_2d=False)
```

File /opt/anaconda3/envs/tensorflow/lib/python3.10/site-packages/sklearn/utils/multiclass.py:270, in type_of_target(y)

```
265 valid = (
266     isinstance(y, Sequence) or issparse(y) or hasattr(y, "__array__")
267 ) and not isinstance(y, str)
268 if not valid:
--> 270     raise ValueError(
271         "Expected array-like (array or non-string sequence), got %r" % y
272     )
273 sparse_pandas = y.__class__.__name__ in ["SparseSeries", "SparseArray"]
274 if sparse_pandas:
```

ValueError: Expected array-like (array or non-string sequence), got
<keras.preprocessing.image.DirectoryIterator object at 0x1833ef8b0>

